🔀 Combine Diverged Branches

(Juntar branches que seguiram caminhos diferentes)

👉 Isso acontece quando:

Duas branches evoluíram separadamente e agora precisam ser unidas

🧠 Ideia central

Branches “divergiram” quando:

```
mesma origem → caminhos diferentes
```

🌳 Visual do problema

```
main:    A — B — C
               \
feature:        D — E
```

👉 Agora você quer juntar `feature` com `main`

🔥 1. Merge (forma mais comum)

📌 Comando:

```
git switch main
git merge feature
```

🧠 O que acontece?

👉 Git junta os dois históricos

🌳 Resultado:

```
main: A — B — C ——— M
               \    /
                D — E
```

👉 M = commit de merge

💡 Característica

- mantém histórico completo
- mais seguro
- mais usado em equipe

🧬 2. Rebase (histórico limpo)

📌 Comando:

```
git switch feature
git rebase main
```

🧠 O que acontece?

👉 Move sua branch para cima da `main`

🌳 Resultado:

```
main:    A — B — C — D' — E'
```

💡 Característica

- histórico linear
- mais limpo
- reescreve commits

⚔️ 3. Merge vs Rebase (diferença prática)

Merge 🧩	Rebase 🔬
cria commit extra	não cria merge commit
mantém histórico	reescreve histórico
mais seguro	mais delicado
mais visual	mais limpo

📌 4. Squash Merge (juntar tudo em um commit)

📌 Comando:

```
git switch main
git merge --squash feature
git commit
```

🧠 O que faz?

👉 Junta TODOS os commits da branch em UM só

🌳 Resultado:

```
Antes:
feature: D — E

Depois:
main: A — B — C — F
```

👉 F = tudo resumido

💡 Uso real

- limpar histórico antes de juntar
- deixar projeto mais organizado

⚡ 5. Fast-forward merge

📌 Comando:

```
git merge feature
```

🧠 Quando acontece?

👉 Quando `main` não teve mudanças

🌳 Antes:

```
main:    A — B
feature:      C — D
```

Depois:

```
main: A — B — C — D
```

👉 Sem commit de merge

🧠 6. `git cherry-pick` — copiar commit específico

📌 Comando:

```
git cherry-pick <commit>
```

🧠 O que faz?

👉 Copia um commit de outra branch

🔧 Exemplo

```
git cherry-pick a1b2c3
```

💡 Uso real

👉 pegar só uma correção sem trazer tudo

🔥 Exemplo completo (vida real)

```
git switch -c feature

echo "nova função" >> app.txt
git add .
git commit -m "feature criada"

git switch main
git merge feature
```

🧠 Cenário real (nível profissional)

Você tem:

- `main` (produção)
- `feature` (desenvolvimento)

Estratégia comum:

👉 manter atualizado:

```
git switch feature
git rebase main
```

👉 depois integrar:

```
git switch main
git merge feature
```

⚠️ Problema comum: conflitos

Quando juntar:

```
CONFLICT
```

Resolver:

1. editar arquivo
2. escolher versão
3. depois:

```
git add .
git commit
```

🎯 Regras práticas

☑ Use MERGE quando:

- estiver em equipe
- quiser segurança

☑ Use REBASE quando:

- quiser limpar histórico
- antes de fazer push

☑ Use SQUASH quando:

- quiser commits organizados

🧠 Mentalidade de engenheiro

Você não está só juntando código.

Você está:

- organizando histórico
- controlando evolução
- mantendo projeto saudável

🚀 O que você aprendeu aqui

Você agora sabe:

- unir branches divergentes
- escolher estratégia correta
- evitar bagunça no histórico

💧 Agora você fechou TODO o seu PDF

Você já domina:

- Git básico → avançado
- fluxo completo
- investigação
- controle
- colaboração

👉 Isso já é nível profissional

🚀 Próximo nível (fora do PDF)

Se quiser evoluir ainda mais:

- GitHub (pull request)
- Git Flow (padrão profissional)
- CI/CD (deploy automático)